

Finding Similar Items: Locality Sensitive Hashing

Adapted from *Mining of Massive Datasets* book (ch.3) and course
Jure Leskovec, Anand Rajaraman, Jeff Ullman Stanford University
<http://www.mmnds.org>

Hash functions/tables

- **Hash function**: a function that maps input items from a set of variable size (possible infinite) to output items (called *hash codes* or *hashes*) from a set of fixed size
 - Usually a function working on the binary representation of the object
 - **Collisions**: distinct inputs with same hashes
 - **Use**: indexing - search in $O(1)$ in the absence of collisions
 - Popular data structure: *Hash table*
 - **Desirable properties**:
 - Deterministic
 - The set of outputs should be (easily) extensible with the growth of input data
 - Uniform distribution of inputs over outputs/hashes
 - Collisions are minimized and are not systematic
- Locality Sensitive Hashing**
is different:
High chances of collisions for similar data is needed

Outline

- Finding similar documents (Min-Hashing)
- LSH families and amplifiers
- Min-Hash as an LSH for Jaccard distance
- LSH for cosine distance
- LSH for Euclidean distance

Part 1: Finding similar documents

LSH for Jaccard distance

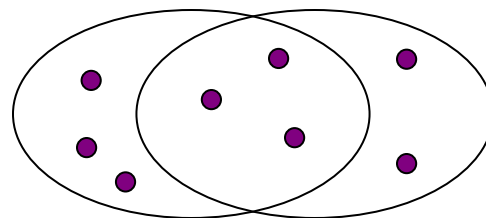
Similar *sets*

- Many problems can be expressed as finding “similar” sets:
- Examples:
 - **Pages with similar words**
 - For duplicate detection, plagiarism, classification by topic
 - **Customers who purchased similar product sets**
 - Or Products with similar customer sets
 - **Images with similar features**

A distance measure for sets

- **Goal: Find near-neighbors in high-dim. space**

- We formally define “near neighbors” as points that are a “small distance” apart
- For each application, we first need to define what “**distance**” means
- **For sets: Jaccard distance/similarity**
 - The **Jaccard similarity** of two **sets** is the size of their intersection divided by the size of their union:
$$\text{sim}(C_1, C_2) = |C_1 \cap C_2| / |C_1 \cup C_2|$$
 - **Jaccard distance:** $d(C_1, C_2) = 1 - |C_1 \cap C_2| / |C_1 \cup C_2|$



3 in intersection

8 in union

Jaccard similarity= 3/8

Jaccard distance = 5/8

Task: Finding Similar Documents

- **Goal:** Given a large number (N in the millions or billions) of documents, find “near duplicate” pairs
- **Applications:**
 - Mirror websites, or approximate mirrors
 - Don’t want to show both in search results
 - Similar news articles at many news sites
 - Cluster articles by “same story”
- **Problems:**
 - Too many documents to compare all pairs
 - Many small pieces of one document can appear out of order in another
 - Documents are so large or so many that they cannot fit in main memory

Motivation for Minhash/LSH

- Suppose we need to find near-duplicate documents among $N = 1$ million documents
- Naively, we would have to compute **similarities** for every pair of docs
 - $N(N - 1)/2 \approx 5 \cdot 10^{11}$ comparisons
 - At 10^5 secs/day and 10^6 comparisons/sec, it would take **5 days**
- For $N = 10$ million, it takes more than a year...

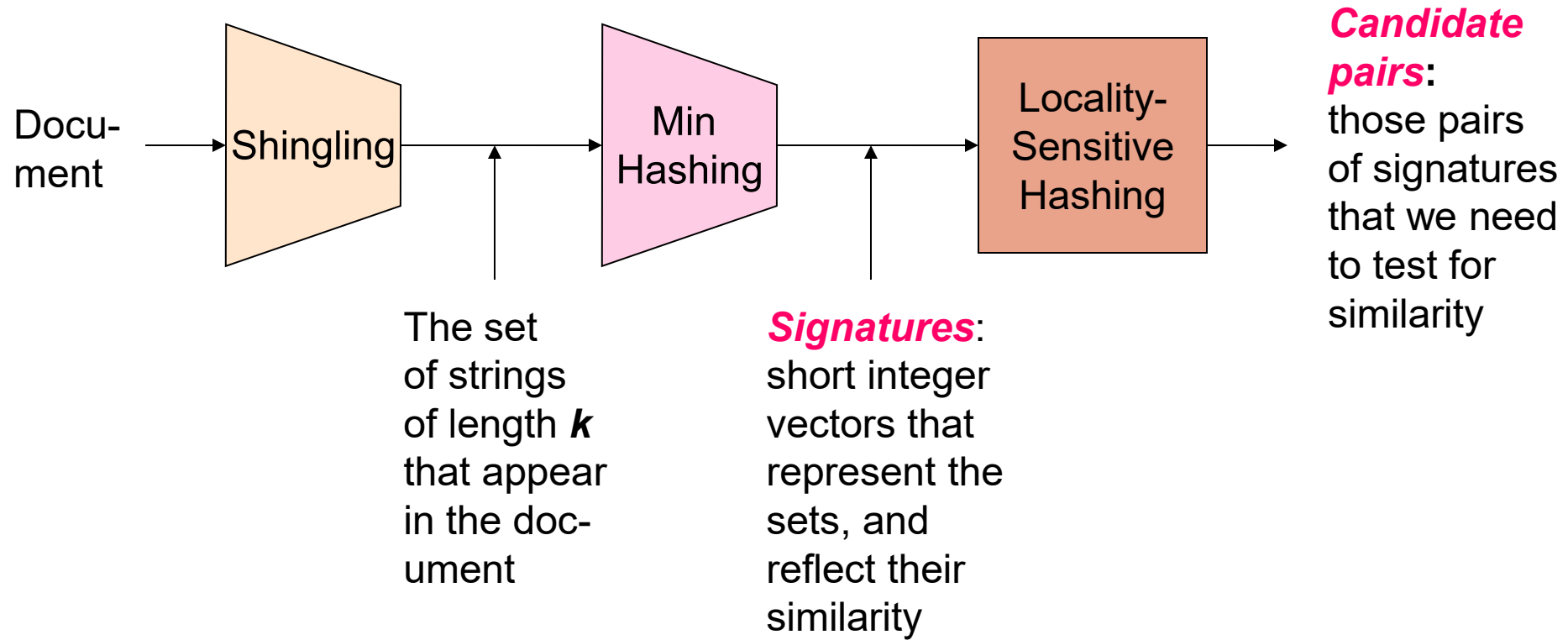
Focus: find similar text documents

- **Main idea: Candidates**
- -- **Pass 1:** Take documents and hash them to buckets such that documents that are similar hash to the same bucket
 - **Benefits:** Instead of $O(N^2)$ comparisons, we need $O(N)$ to find similar documents
- -- **Pass 2:** Only compare documents that are **candidates** (i.e., they hashed to the same bucket)

3 Essential Steps for Similar Docs

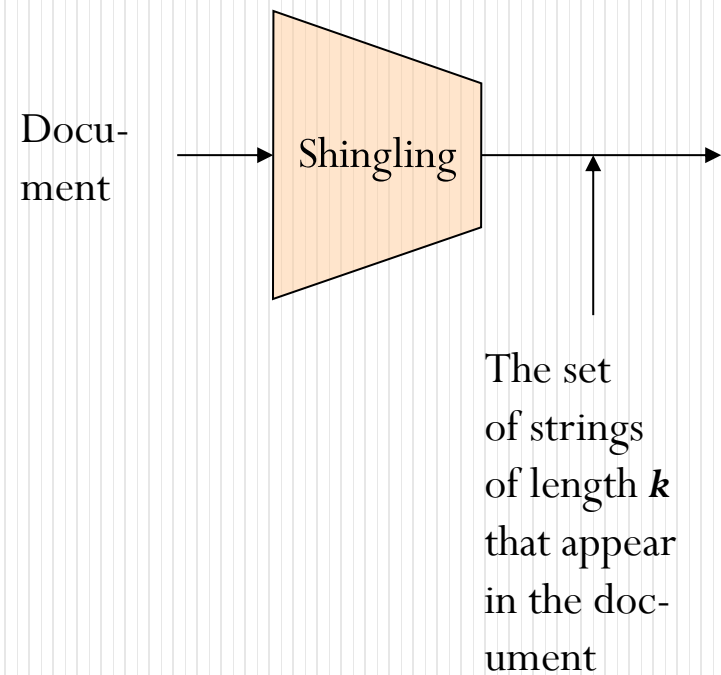
1. *Shingling*: Convert documents to sets
2. *Min-Hashing*: Convert large sets to short signatures, while preserving similarity
3. *Locality-Sensitive Hashing*: Focus on pairs of signatures likely to be from similar documents
 - **Candidate pairs!**

The Big Picture



Shingling

Step 1: *Shingling*: Convert documents to sets



Documents as High-Dim Data

- Step 1: *Shingling*: Convert documents to sets
- Simple approaches:
 - Document = set of words appearing in document
 - Document = set of “important” words
 - Don’t work well for this application. Why?
- Need to account for ordering of words!
- A different way: *Shingles*!

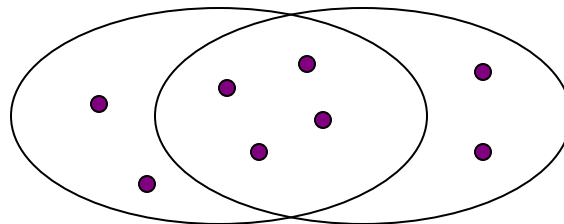
Define: Shingles

- A ***k*-shingle** for a document is a sequence of k tokens that appears in the doc
 - Tokens can be **characters**, **words** or something else, depending on the application
 - Here we assume tokens = characters
- **Example:** $k=2$; document $D_1 = \text{abcab}$
Set of 2-shingles: $S(D_1) = \{\text{ab}, \text{bc}, \text{ca}\}$
 - **Option:** Shingles as a bag (multiset), count **ab** twice: $S'(D_1) = \{\text{ab}, \text{bc}, \text{ca}, \text{ab}\}$

Similarity Metric for Shingles

- **Document D_1 is a set of its k -shingles $C_1=S(D_1)$**
- Equivalently, each document is a 0/1 vector in the space of k -shingles
 - Each unique shingle is a dimension
 - Vectors are very sparse
- **A natural similarity measure is the Jaccard similarity:**

$$sim(D_1, D_2) = |C_1 \cap C_2| / |C_1 \cup C_2|$$

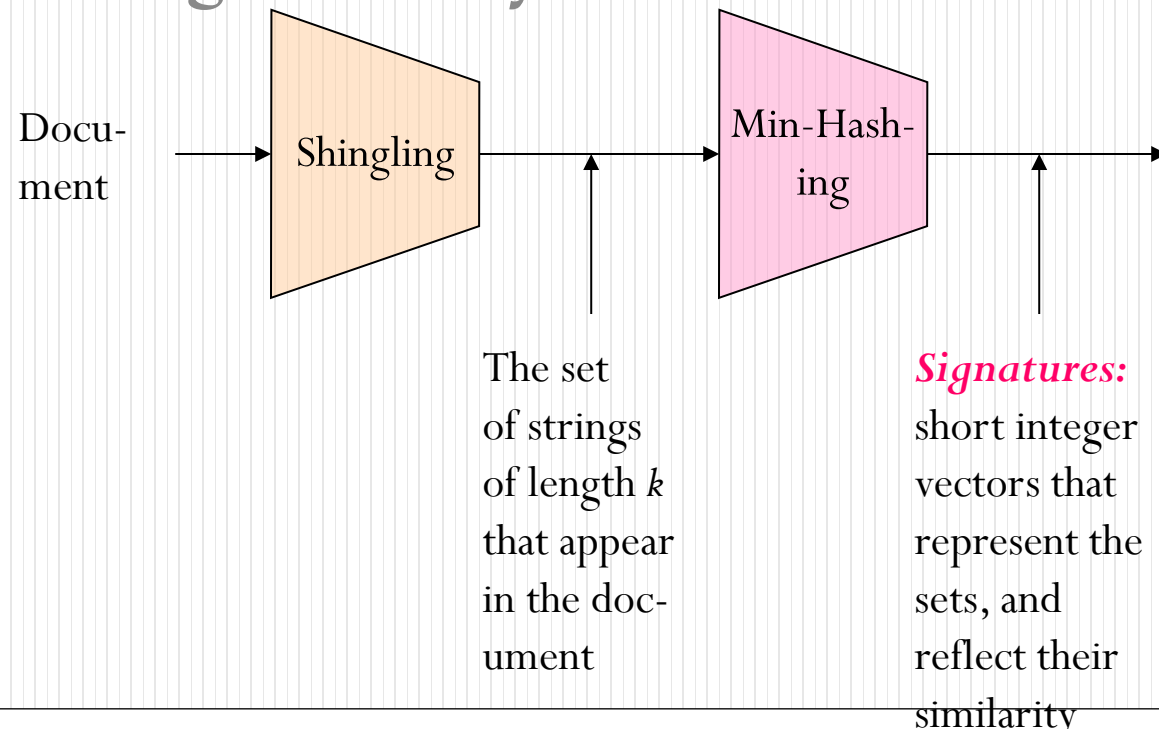


Working Assumption

- Documents that have lots of shingles in common have similar text, even if the text appears in different order
- **Caveat:** You must pick k large enough, or most documents will have most shingles
 - $k = 5$ is OK for short documents
 - $k = 10$ is better for long documents

MinHashing

Step 2: *Minhashing*: Convert large sets to short signatures, while preserving similarity



From Sets to Boolean Matrices

- **Rows** = elements (shingles)
- **Columns** = sets (documents)
 - 1 in row e and column s if and only if e is a member of s
 - Column similarity is the Jaccard similarity of the corresponding sets (rows with value 1)
 - **Typical matrix is sparse!**
- **Each document is a column:**
 - **Example:** $\text{sim}(C_1, C_2) = ?$
 - Size of intersection = 3; size of union = 6, Jaccard similarity (not distance) = $3/6$
 - $d(C_1, C_2) = 1 - (\text{Jaccard similarity}) = 3/6$

		Documents			
Shingles	1	1	1	0	
	1	1	0	1	
	0	1	0	1	
	0	0	0	1	
	1	0	0	1	
	1	1	1	0	
	1	0	1	0	

Hashing Columns (Signatures)

- **Key idea:**

Hash docs into buckets. Expect that “most” pairs of near duplicate docs hash into the same bucket!

- **Goal: Find a hash function $h(\cdot)$ such that:**

- If $\text{sim}(C_1, C_2)$ is high, then with high prob. $h(C_1) = h(C_2)$
- If $\text{sim}(C_1, C_2)$ is low, then with high prob. $h(C_1) \neq h(C_2)$

- **Clearly, the hash function depends on the similarity metric:**

- Not all similarity metrics have a suitable hash function

- **There is a suitable hash function for the Jaccard similarity: It is called **Min-Hashing****

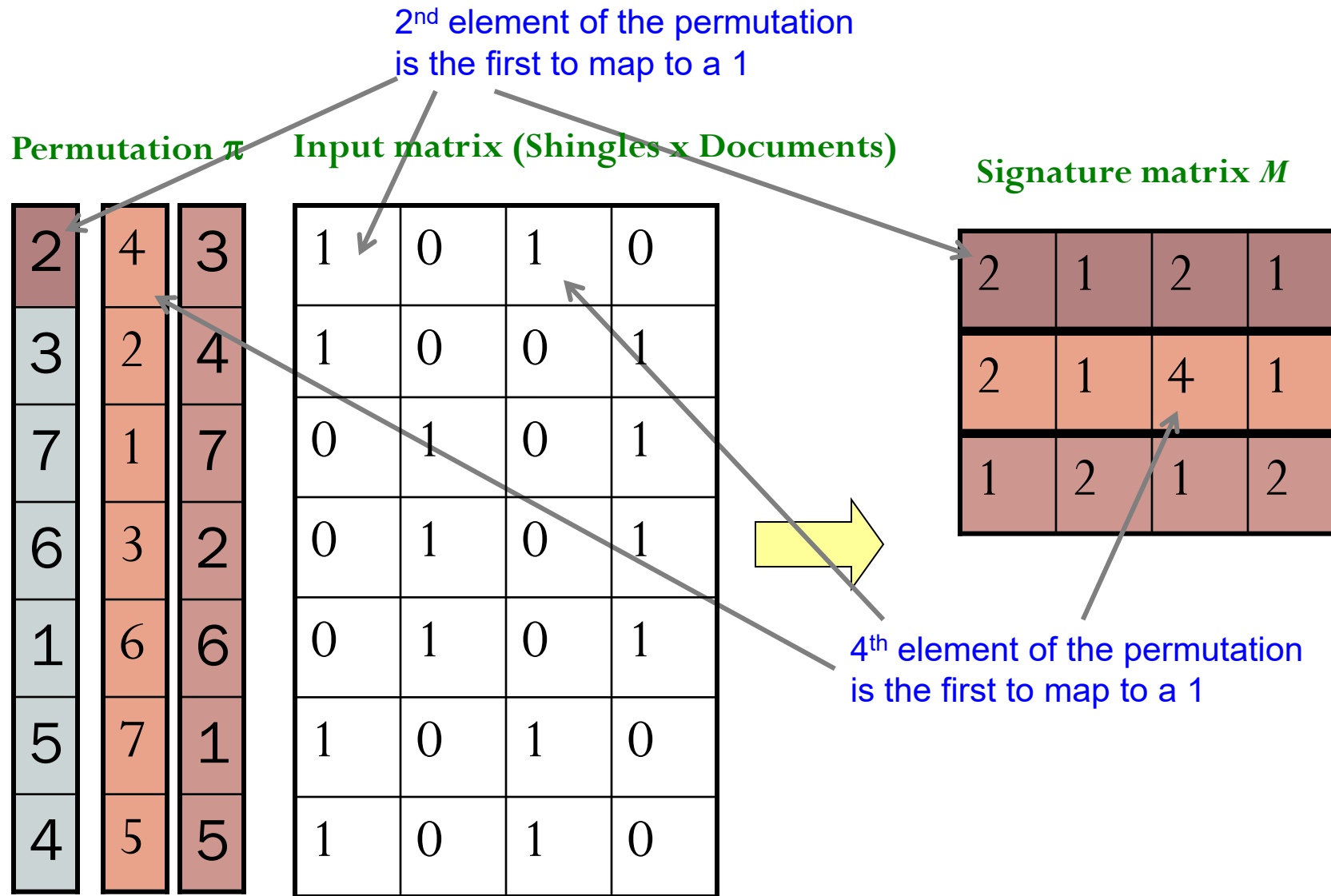
Min-Hashing

- Imagine the rows of the boolean matrix permuted under **random permutation** π
- Define a **“hash” function** $h_{\pi}(C)$ = the index of the **first** (in the permuted order π) row in which column C has value 1:

$$h_{\pi}(C) = \min_{\pi} \pi(C)$$

- Use several (e.g., 100) independent hash functions (that is, permutations) to create a signature of a column

Min-Hashing Example



Four Types of Rows

- Given cols C_1 and C_2 , rows may be classified as:

	$\underline{C_1}$	$\underline{C_2}$
A	1	1
B	1	0
C	0	1
D	0	0

- $a = \#$ rows of type A, etc.
- Note:** $\text{sim}(C_1, C_2) = a / (a + b + c)$
- Then:** $\Pr[h(C_1) = h(C_2)] = \text{Sim}(C_1, C_2)$
 - Look down the cols C_1 and C_2 until we see a 1
 - If it's a type-A row, then $h(C_1) = h(C_2)$
If a type-B or type-C row, then not

Similarity for Signatures

- We know: $\Pr[h_\pi(C_1) = h_\pi(C_2)] = \text{sim}(C_1, C_2)$
- Now generalize to multiple hash functions
- The *similarity of two signatures* is the fraction of the hash functions in which they agree
- **Note:** Because of the Min-Hash property, the similarity of columns is the same as the expected similarity of their signatures

Min-Hashing Example

Permutation π

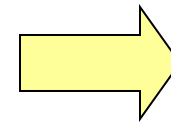
2	4	3
3	2	4
7	1	7
6	3	2
1	6	6
5	7	1
4	5	5

Input matrix (Shingles x Documents)

1	0	1	0
1	0	0	1
0	1	0	1
0	1	0	1
0	1	0	1
1	0	1	0
1	0	1	0

Signature matrix M

2	1	2	1
2	1	4	1
1	2	1	2



Similarities:

Col/Col

Sig/Sig

	1-3	2-4	1-2	3-4
Col/Col	0.75	0.75	0	0
Sig/Sig	0.67	1.00	0	0

Min-Hash Signatures

- **Pick $K=100$ random permutations of the rows**
- Think of $\text{sig}(\mathbf{C})$ as a column vector
- $\text{sig}(\mathbf{C})[i] =$ according to the i -th permutation, the index of the first row that has a 1 in column C

$$\text{sig}(\mathbf{C})[i] = \min (\pi_i(\mathbf{C}))$$

- **Note:** The sketch (signature) of document C is small **~ 100 bytes!**
- **We achieved our goal!** We “compressed” long bit vectors into short signatures

Implementation Trick

- **Permuting rows even once is prohibitive**
- **Row hashing!**
 - Pick $K = 100$ hash functions h_i
 - Ordering under h_i gives a random row permutation!
- **One-pass implementation**
 - For each column C and hash-func. h_i keep a “slot” for the min-hash value
 - Initialize all $sig(C)[i] = \infty$
 - **Scan rows looking for 1s**
 - Suppose row j has 1 in column C
 - Then for each k_i :
 - If $k_i(j) < sig(C)[i]$, then $sig(C)[i] \leftarrow k_i(j)$

How to pick a random hash function $h(x)$?

Universal hashing:

$h_{a,b}(x) = ((a \cdot x + b) \bmod p) \bmod N$
where:

a, b ... random integers

p ... prime number ($p > N$)

Example

Row	C ₁	C ₂
1	1	0
2	0	1
3	1	1
4	1	0
5	0	1

$$h(x) = x \bmod 5$$

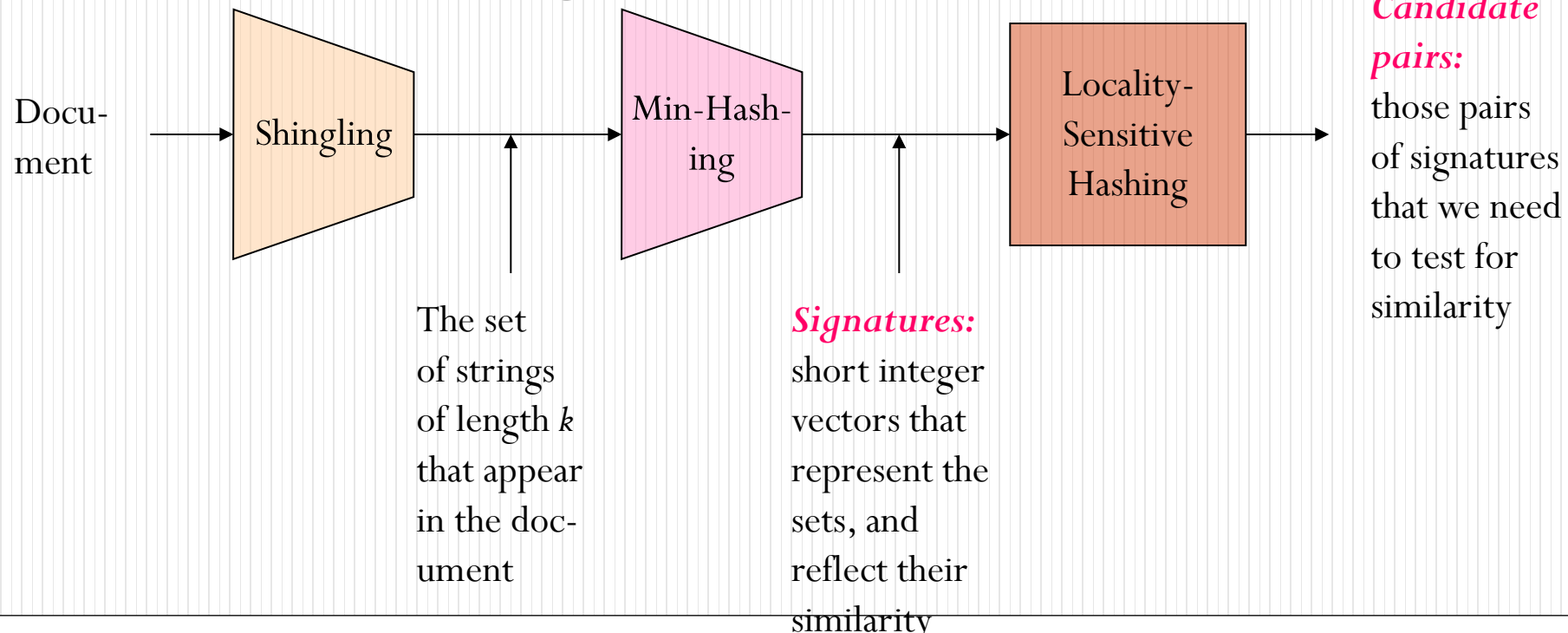
$$g(x) = (2x+1) \bmod 5$$

	Sig1	Sig2
$h(1) = 1$	1	∞
$g(1) = 3$	3	∞
$h(2) = 2$	1	2
$g(2) = 0$	3	0
$h(3) = 3$	1	2
$g(3) = 2$	2	0
$h(4) = 4$	1	2
$g(4) = 4$	2	0
$h(5) = 0$	1	0
$g(5) = 1$	2	0

Locality Sensitive Hashing

Step 3: *Locality-Sensitive Hashing*:

Focus on pairs of signatures likely to be from similar documents



Candidates from Min-Hash

2	1	4	1
1	2	1	2
2	1	2	1

- **Goal:** Find documents with Jaccard similarity at least s (for some similarity threshold, e.g., $s=0.8$)
- **Pick a similarity threshold s ($0 < s < 1$)**
- Columns x and y of M are a **candidate pair** if their signatures agree on at least fraction s of their rows:
 - We expect documents x and y to have the same (Jaccard) similarity as their signatures

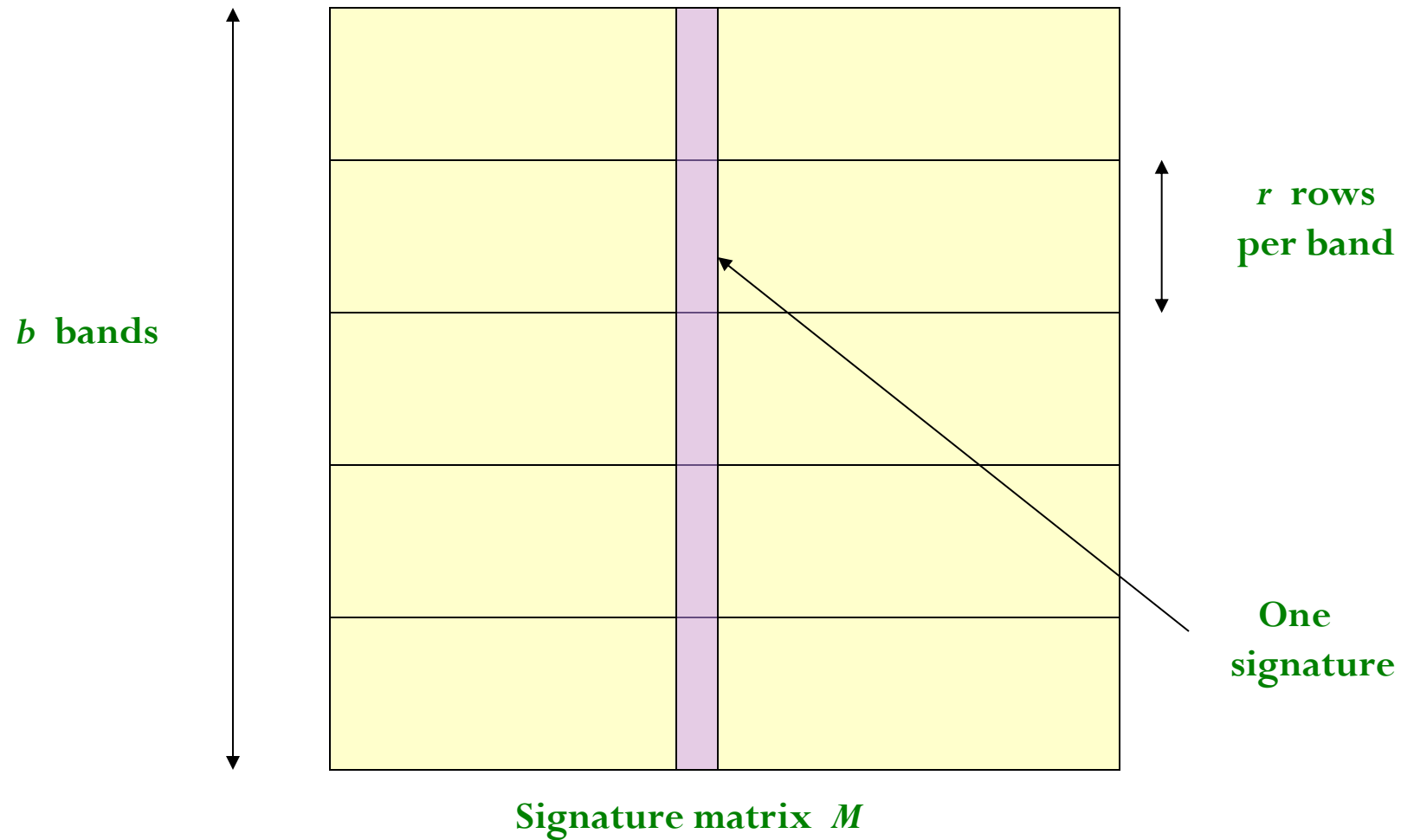
LSH for Min-Hash

2	1	4	1
1	2	1	2
2	1	2	1

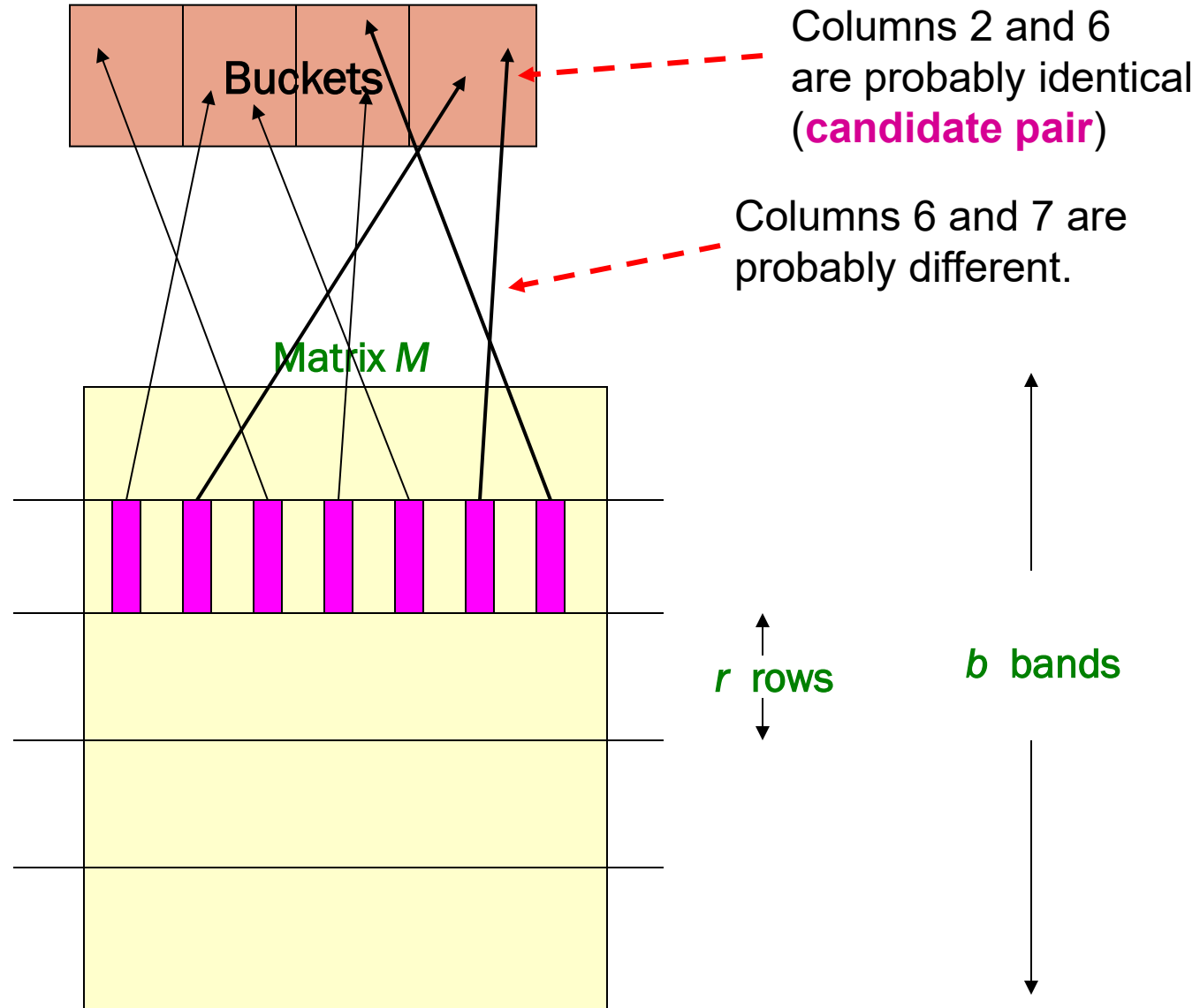
- **Big idea:** Hash columns of signature matrix M several times
- Arrange that (only) **similar columns** are likely to **hash to the same bucket**, with high probability
- **Candidate pairs are those that hash to the same bucket at least once**

Partition M into b Bands

2	1	4	1
1	2	1	2
2	1	2	1



Hashing Bands



Partition M into Bands

- Divide matrix M into b bands of r rows
- For each band, hash its portion of each column to a hash table with k buckets
 - Make k as large as possible
- *Candidate* column pairs are those that hash to the same bucket for ≥ 1 band
- Tune b and r to catch most similar pairs, but few non-similar pairs

Simplifying Assumption

- There are **enough buckets** that columns are unlikely to hash to the same bucket unless they are **identical** in a particular band
- Hereafter, we assume that “**same bucket**” means “**identical in that band**”
- Assumption needed only to simplify analysis, not for correctness of algorithm

C_1, C_2 are 80% Similar

- Find pairs of $\geq s=0.8$ similarity, set $b=20$, $r=5$ (signature length=100)
- **Assume:** $\text{sim}(C_1, C_2) = 0.8$
 - Since $\text{sim}(C_1, C_2) \geq s$, we want C_1, C_2 to be a **candidate pair**: We want them to hash to at **least 1 common bucket** (at least one band is identical)
- **Probability C_1, C_2 identical in one particular band:** $(0.8)^5 = 0.328$
- Probability C_1, C_2 are **not** identical in any of the 20 bands: $(1-0.328)^{20} = 0.00035$
 - i.e., about 1 / 3000th of the 80%-similar column pairs are **false negatives** (we miss them)
 - We would find 99.965% pairs of truly similar documents

C_1, C_2 are 30% Similar

- Find pairs of $\geq s=0.8$ similarity, set $b=20, r=5$
- **Assume:** $\text{sim}(C_1, C_2) = 0.3$
 - Since $\text{sim}(C_1, C_2) < s$ we want C_1, C_2 to hash to **NO common buckets** (all bands should be different)
- **Probability C_1, C_2 identical in one particular band:** $(0.3)^5 = 0.00243$
- Probability C_1, C_2 identical in at least 1 of 20 bands: $1 - (1 - 0.00243)^{20} = 0.0474$
 - In other words, approximately 4.74% pairs of docs with similarity 0.3% end up becoming **candidate pairs**
 - They are **false positives** since we will have to examine them (they are candidate pairs) but then it will turn out their similarity is below threshold s

LSH Involves a Tradeoff

- **Pick:**
 - The number of Min-Hashes (rows of M)
 - The number of bands b , and
 - The number of rows r per bandto balance false positives/negatives
- **Example:** If we had only 15 bands of 5 rows, the number of false positives would go down, but the number of false negatives would go up

b bands, r rows/band

- Columns C_1 and C_2 have similarity t
- Pick any band (r rows)
 - Prob. that all rows in band equal = t^r
 - Prob. that some row in band unequal = $1 - t^r$
- Prob. that no band identical = $(1 - t^r)^b$
- Prob. that at least 1 band identical = $1 - (1 - t^r)^b$

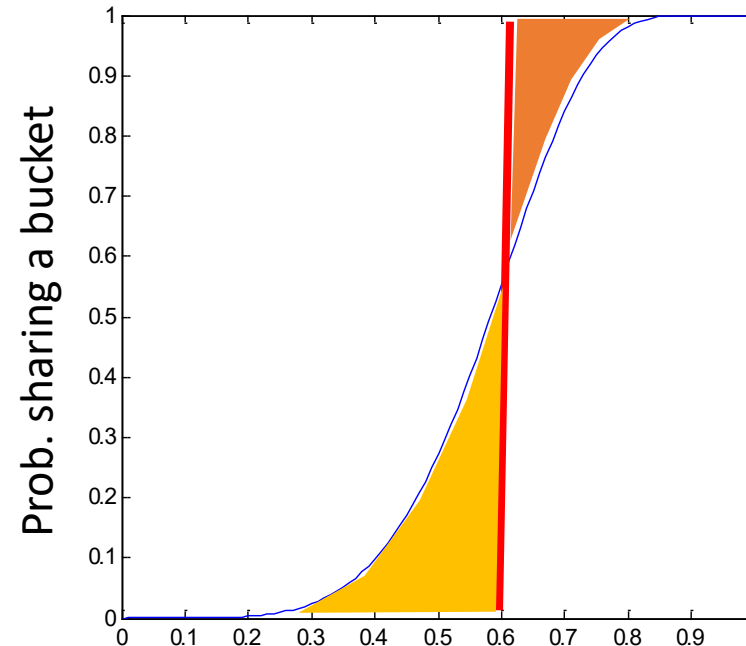
Example: $b = 20$; $r = 5$

- **Similarity s**
- **Prob. that at least 1 band is identical:**

s	$1-(1-s^r)^b$
.2	.006
.3	.047
.4	.186
.5	.470
.6	.802
.7	.975
.8	.9996

Picking r and b : The S-curve

- Picking r and b to get the best S-curve
 - 50 hash-functions ($r=5$, $b=10$)



Orange area: False Negative rate

Yellow area: False Positive rate

Summary: 3 Steps

- **Shingling:** Convert documents to sets
 - We used hashing to assign each shingle an ID
- **Min-Hashing:** Convert large sets to short signatures, while preserving similarity
 - We used **similarity preserving hashing** to generate signatures with property $\Pr[h_{\pi}(C_1) = h_{\pi}(C_2)] = \text{sim}(C_1, C_2)$
 - We used hashing to get around generating random permutations
- **Locality-Sensitive Hashing:** Focus on pairs of signatures likely to be from similar documents
 - We used hashing to find **candidate pairs** of similarity $\geq s$

Part 2: Locality sensitive hashing families

Amplifying hash functions

Distance measures

- $d(\cdot)$ is a distance metric if it is a function from pairs of points x, y to real numbers such that:
 - $d(x, y) \geq 0$
 - $d(x, y) = 0$ iff $x = y$
 - $d(x, y) = d(y, x)$
 - $d(x, y) \leq d(x, z) + d(z, y)$ (triangle inequality);
- Jaccard distance for sets = $1 - \text{Jaccard similarity}$
- Cosine distance for vectors = angle between the vectors
- Minkowski distances:
 - L2 norm (Euclidean): $d(x, y)$ = square root of the sum of the squares of the differences between x and y in each dimension
 - L1 norm (Manhattan): sum of absolute value of the differences in each dimension

Locality sensitive hashing

- Uses hash functions that hash similar input items to identical values (the same "buckets") with high probability $\rightarrow$ preserves locality
- The function is strongly related to the similarity/distance function in the input space

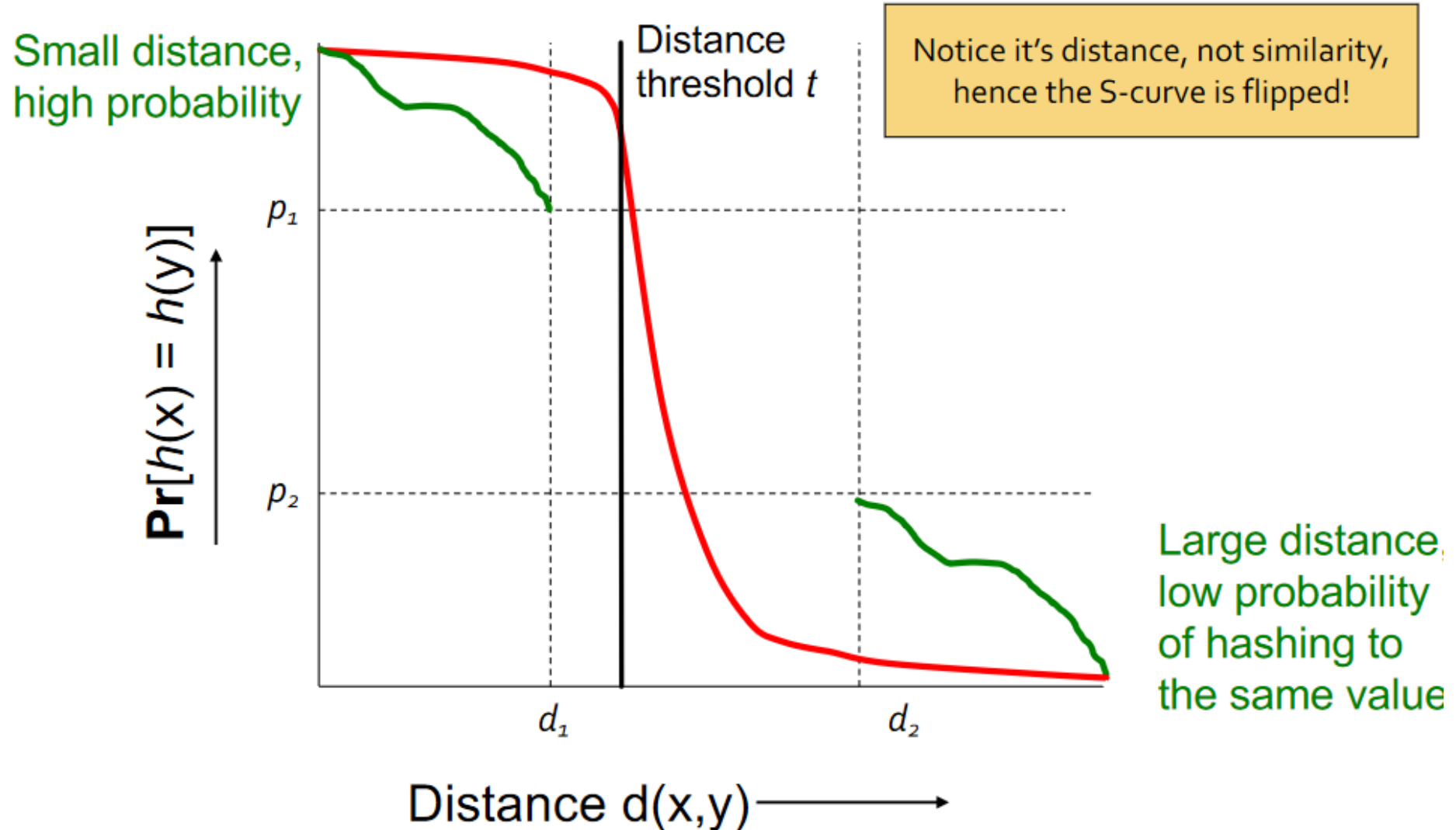
- Suppose we have a space S of points with a **distance measure** $d(x,y)$.

A family H of hash functions is said to be (d_1, d_2, p_1, p_2) -sensitive if for any x and y in S :

- 1.If $d(x, y) < d_1$, then the probability that $h(x) = h(y)$ is at least p_1 for all h in H
- 2.If $d(x, y) > d_2$, then the probability that $h(x) = h(y)$ is at most p_2 for all h in H

(Interesting when $d_1 < d_2$ and $p_1 > p_2$)

(d1, d2, p1, p2)-sensitive



MinHash as an LS family

- Let:
 - S = space of all sets,
 - d = Jaccard distance,
 - H the family of Min-Hash functions for all permutations of rows

Then for any hash function h in H : $\Pr[h(x) = h(y)] = 1 - d(x, y)$

Min-Hashing gives a $(d_1, d_2, (1-d_1), (1-d_2))$ -sensitive family for any $d_1 < d_2$

- Min-hash is a $(1/3, 2/3, 2/3, 1/3)$ -sensitive family for S and d .
- If distance $< 1/3$ (similarity $\geq 2/3$) then probability that Min-Hash values agree is $\geq 2/3$

Amplifying an LS family

- Can we reproduce the “S-curve” effect we saw before for any LS family?
- The “bands” technique we learned for signature matrices carries over to this more general setting
- Can do LSH with any (d_1, d_2, p_1, p_2) -sensitive family
- Two constructions:
 - AND construction like “rows in a band”
 - OR construction like “many bands”

AND of hash functions

- Given family H , construct family H' consisting of r functions from H

For $h' = [h_1, \dots, h_r]$ in H' , we say $h'(x) = h'(y)$ if and only if $h_i(x) = h_i(y)$ for all i

Note: this corresponds to creating a band of size r

- Theorem: If H is (d_1, d_2, p_1, p_2) -sensitive, then H' is $(d_1, d_2, (p_1)^r, (p_2)^r)$ -sensitive

Proof: Use the fact that h_i 's are independent


Bad Good

OR of hash functions

- Given family H , construct family H' consisting of b functions from H

For $h' = [h_1, \dots, h_b]$ in H' , we say $h'(x) = h'(y)$ if and only if $h_i(x) = h_i(y)$ for at least one i

- Theorem: If H is (d_1, d_2, p_1, p_2) -sensitive, then H' is $(d_1, d_2, 1-(1-p_1)^b, 1-(1-p_2)^b)$ -sensitive

Proof: Use the fact that h_i 's are independent

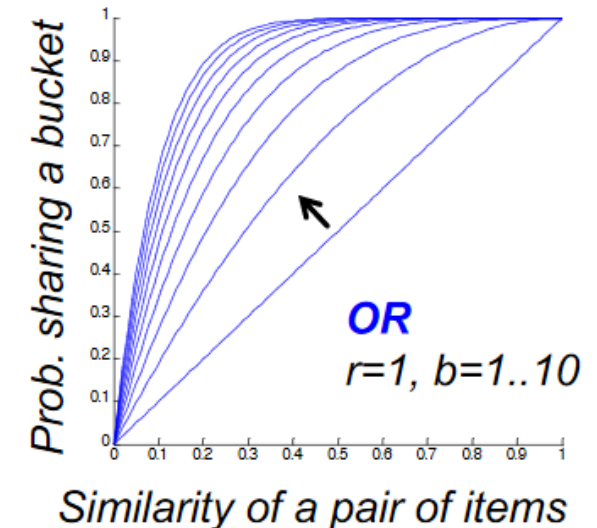
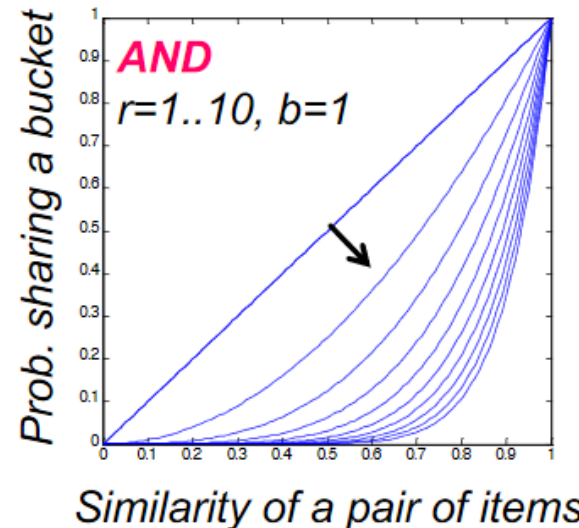
↑
Good

↑
Bad

Effect of AND and OR constructions

- AND makes all probabilities shrink, but by choosing r correctly, we can make the lower prob. approach 0 while the higher does not
- OR makes all probabilities grow, but by choosing b correctly, we can make the higher prob. approach 1 while the lower does not

- By choosing r and b correctly, we can make the lower probability approach 0 while the higher approaches 1

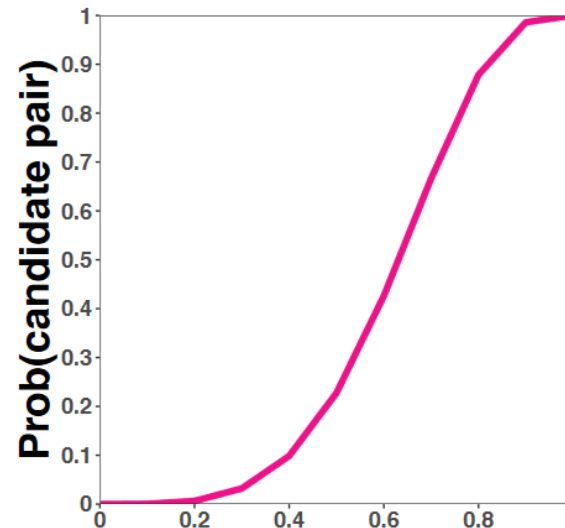


- As for the signature matrix, we can use the AND construction followed by the OR (or vice-versa, or any sequence of AND's and OR's alternating)

AND-OR construction

- Take points x and y s.t. $\Pr[h(x) = h(y)] = s \rightarrow H$ will make (x,y) a candidate pair with prob. s
- AND r times, OR b times $\rightarrow$ makes (x,y) a candidate pair with probability $1-(1-s^r)^b$
- Example:
 - Take H and construct H' by the AND construction with $r=4$. Then, from H' , construct H'' by the OR construction with $b=4$

s	$p=1-(1-s^4)^4$
.2	.0064
.3	.0320
.4	.0985
.5	.2275
.6	.4260
.7	.6666
.8	.8785
.9	.9860

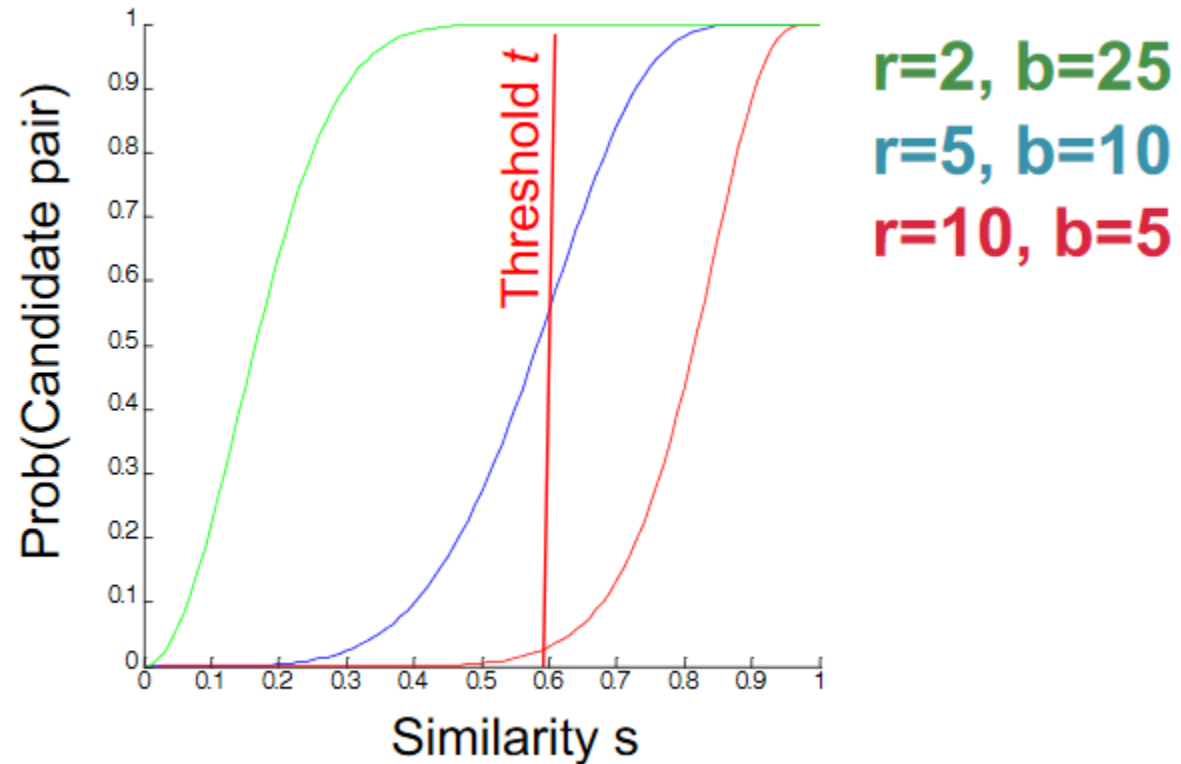


$r = 4, b = 4$

transforms a $(.2, .8, .8, .2)$ -sensitive family
into a $(.2, .8, .8785, .0064)$ -sensitive family

Picking r and b : the S curve

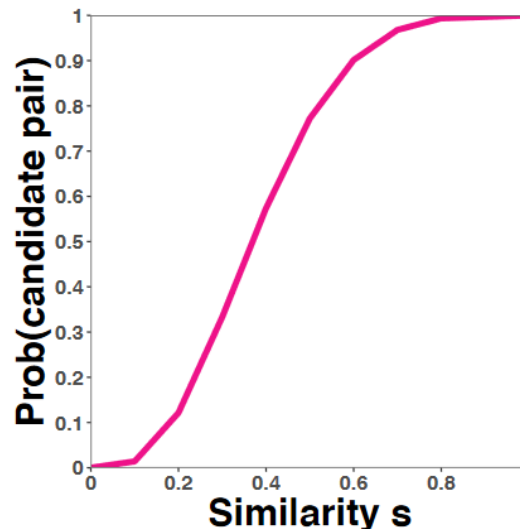
- 50 hash functions ($r*b=50$)



OR-AND construction

- Take points x and y s.t. $\Pr[h(x) = h(y)] = s \rightarrow H$ will make (x,y) a candidate pair with prob. s
- OR b times, AND r times $\rightarrow$ makes (x,y) a candidate with probability $(1-(1-s)^b)^r$
- Example:
 - Take H and construct H' by the OR construction with $b=4$. Then, from H' , construct H'' by the AND construction with $r=4$.

s	$p=(1-(1-s)^4)^4$
.1	.0140
.2	.1215
.3	.3334
.4	.5740
.5	.7725
.6	.9015
.7	.9680
.8	.9936



$r = 4, b = 4$
transforms a $(.2, .8, .8, .2)$ -sensitive family
into a $(.2, .8, .9936, .1215)$ -sensitive family

Cascading constructions

- Example: Apply the (4,4) OR-AND construction followed by the (4,4) AND-OR construction
- Transforms a (.2, .8, .8, .2)-sensitive family into a (.2, .8, .9999996, .0008715)-sensitive family
- Note this family uses 256 ($=4*4*4*4$) of the original hash functions

Summary

- Pick any two distances $d_1 < d_2$
- Start with a $(d_1, d_2, (1-d_1), (1-d_2))$ -sensitive family
- Apply constructions to amplify and obtain (d_1, d_2, p_1, p_2) -sensitive family, where p_1 is almost 1 and p_2 is almost 0.
- The closer to 0 and 1 we get, the more hash functions must be used!

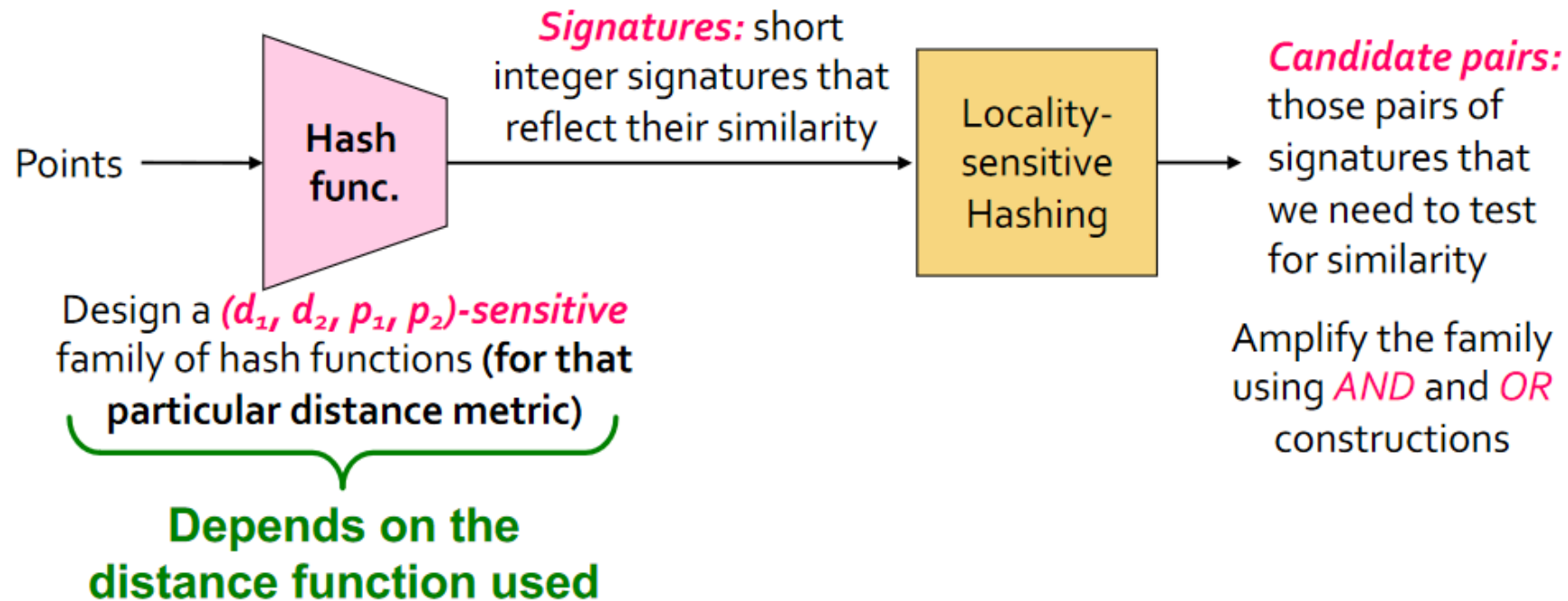
Part 3: Find similar data in numerical spaces

LSH for cosine distance

LSH for Euclidean distance

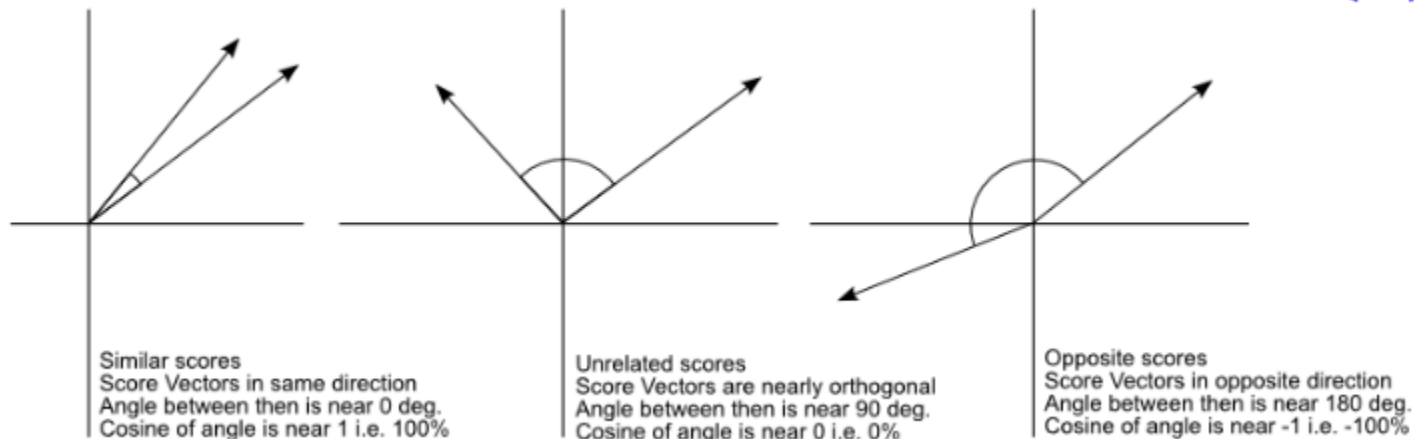
LSH for numerical data

- LSH methods for other distance metrics:
 - Cosine distance: Random hyperplanes
 - Euclidean distance: Projections on lines



Cosine distance

- Cosine distance = angle between vectors from the origin to the points in question
 - $d(A, B) = \arccos(A \times B / (\|A\| \cdot \|B\|))$
 - Has range $[0, \pi]$ (equivalently $[0, 180^\circ]$)
 - Can divide by π to have distance in range $[0, 1]$
- Cosine similarity
 - Defined as $\cos(\theta) = A \times B / (\|A\| \cdot \|B\|)$
 - Has range $-1 \dots 1$ for general vectors; range $0 \dots 1$ for non-negative vectors (angles up to 90°)

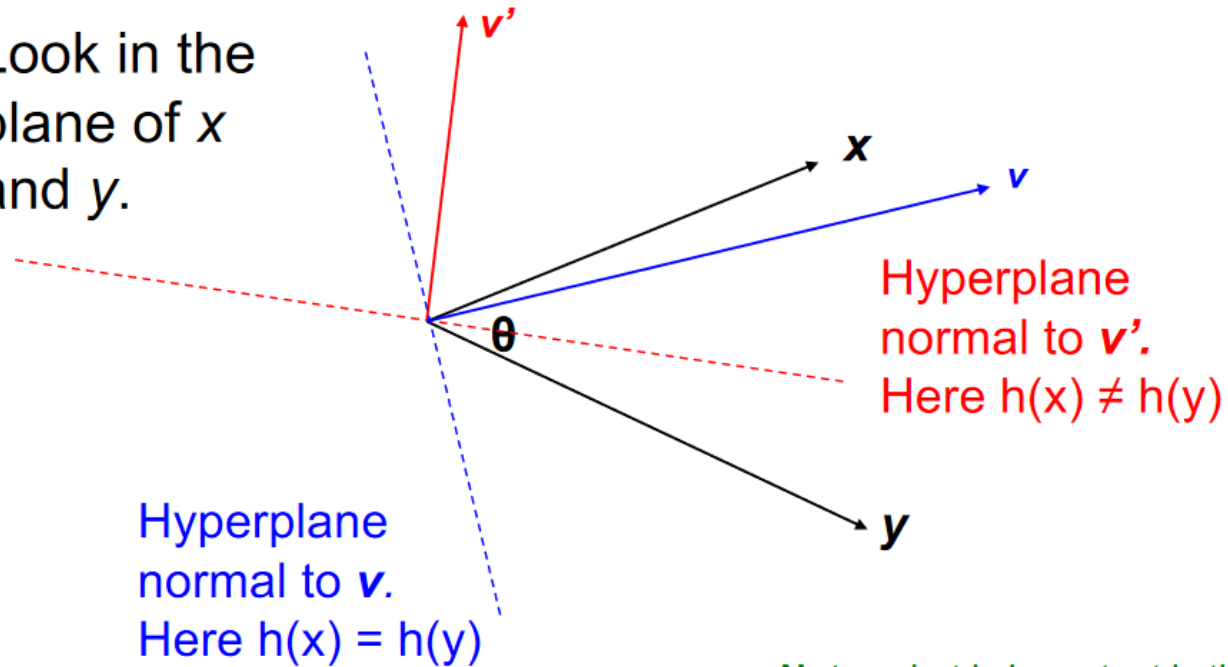


LSH for cosine distance

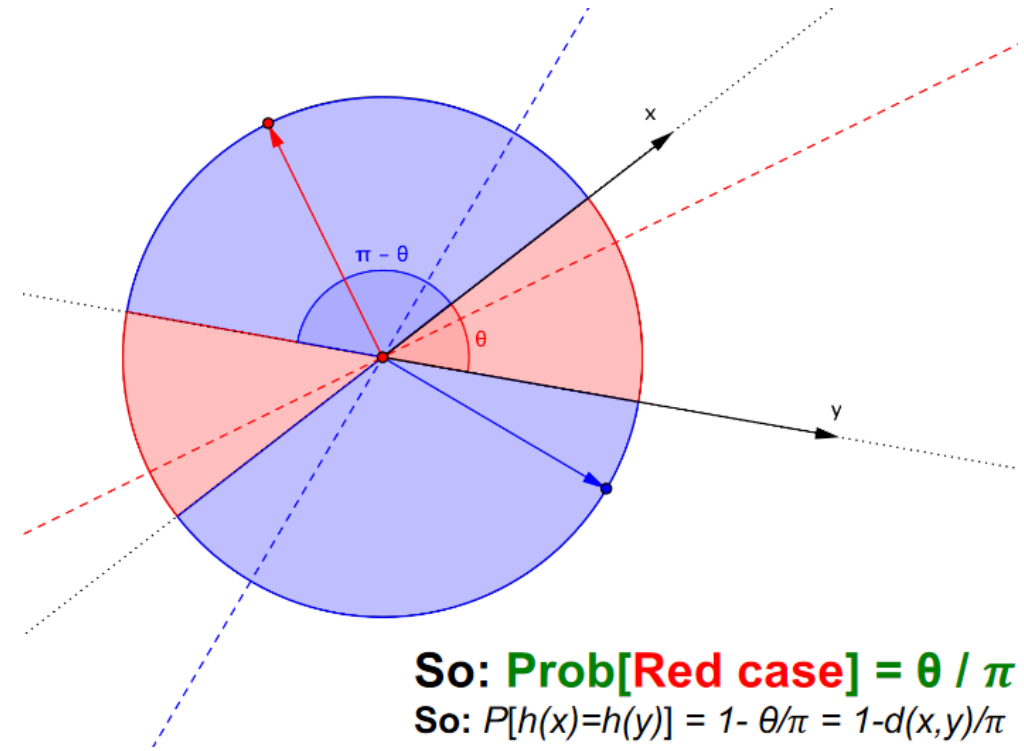
- The technique is called Random Hyperplanes
 - similar to Min-Hashing
- Each vector v determines a hash function h_v with two buckets
 - $h_v(x) = +1$ if $v \times x \geq 0$; $= -1$ if $v \times x < 0$
 - LS-family H = set of all functions derived from any vector
 - Claim: For points x and y , $\Pr[h(x) = h(y)] = 1 - d(x, y) / \pi$
- Random Hyperplanes provides a $(d_1, d_2, (1 - d_1 / \pi), (1 - d_2 / \pi))$ -sensitive family for any d_1 and d_2
 - If $d(x, y) < d_1$, then prob. that $h(x) = h(y)$ is at least $(1 - d_1 / \pi)$.
 - If $d(x, y) > d_2$, then prob. that $h(x) = h(y)$ is at most $(1 - d_2 / \pi)$.

For points x and y , $\Pr[h(x) = h(y)] = 1 - d(x,y) / \pi$

Look in the
plane of x
and y .



Note: what is important is that
hyperplane is outside the angle,
not that the vector is inside.

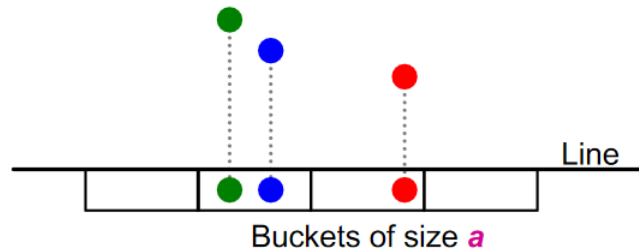


Signature for cosine distance

- Pick some number of random vectors, and hash your data for each vector
- The result is a signature(sketch) of $+1$'s and -1 's for each data point
- Can be used for LSH like we used the Min-Hash signatures for Jaccard distance
- Amplify using AND/OR constructions
- Expensive to pick a random vector in M dimensions for large M
 - Would have to generate random numbers
- It suffices to consider only vectors v consisting of $+1$ and -1 components
 - Assuming data is random, then vectors of $+/-1$ cover the entire space evenly, and does not bias in any way

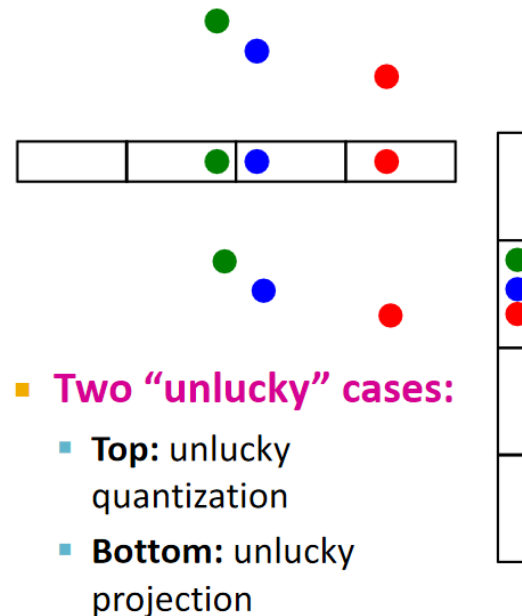
LSH for Euclidean distance

- Hash functions correspond to lines partitioned into buckets of size a
 - Hash each point to the bucket containing its projection onto the line
 - An element of the “Signature” is a bucket id for that given projection line
 - Nearby points are always close; distant points are rarely in same bucket

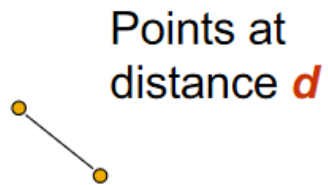


■ **“Lucky” case:**

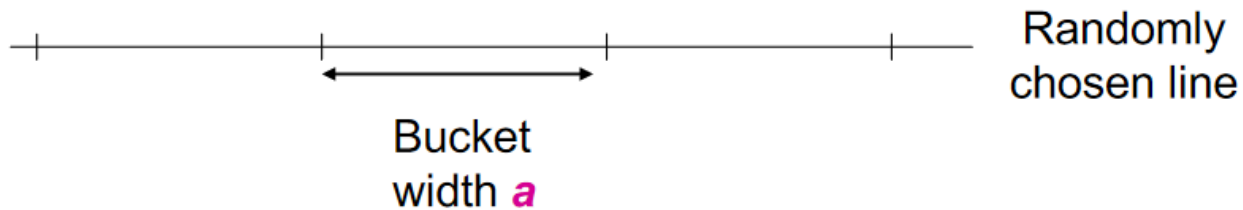
- Points that are close hash in the same bucket
- Distant points end up in different buckets



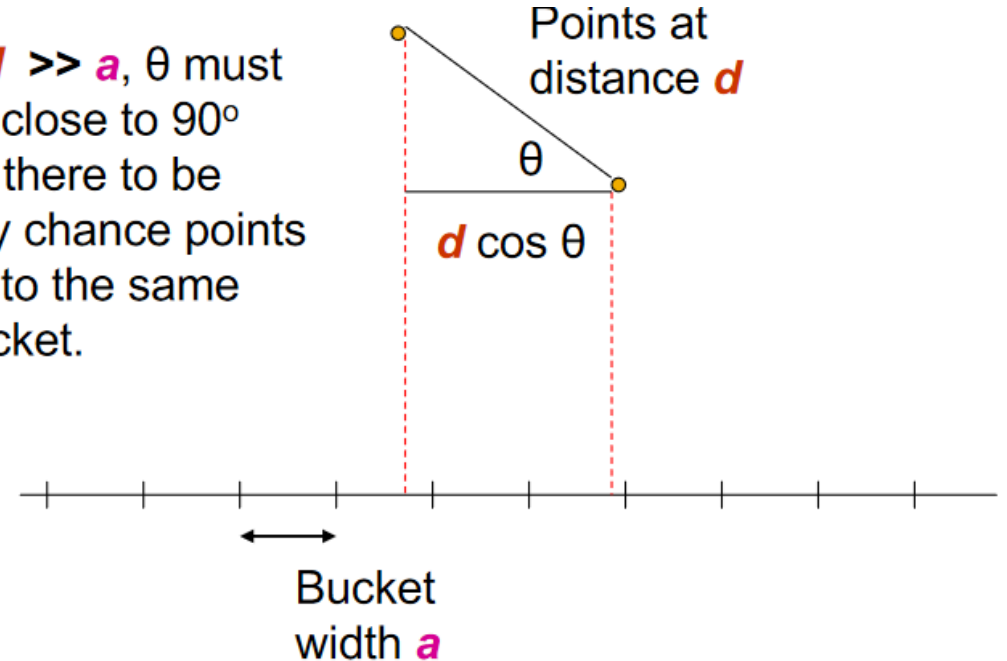
Projection of points



If $d \ll a$, then the chance the points are in the same bucket is at least $1 - d/a$.



If $d \gg a$, θ must be close to 90° for there to be any chance points go to the same bucket.



An LS family for the Euclidean distance

- If points are at distance $d < a/2$, prob. they are in same bucket $\geq 1 - d/a = 1/2$
- If points are at distance $d > 2a$ apart, then they can be in the same bucket only if $d \cos \theta \leq a$
 - $\cos \theta \leq 1/2$
 - $60^\circ < \theta < 90^\circ$, i.e., at most $1/3$
- Yields a **$(a/2, 2a, 1/2, 1/3)$ -sensitive family** of hash functions for any a
- Amplify using AND-OR cascades

Summary

